

Персистентные структуры данных

Цель семинара - научиться сохранять старые версии структуры данных, разделяя между версиями неизменяемые узлы. Сначала мы реализуем стек, затем массив на дереве отрезков и используем этот массив для очереди.

Задачи 1-4 выполняются на семинаре; задача 5 - домашняя.

Нужны односвязные списки, указатели, рекурсия и базовое дерево отрезков. Язык: C++17. Во всех задачах вход корректен, $1 \leq q \leq 200000$, а хранимые значения - целые числа от -10^9 до 10^9 . Остальные ограничения указаны в задачах. Ответы однозначны.

В персистентных задачах действуют общие правила:

- Начальная версия имеет номер 0.
- Каждая изменяющая команда создаёт новую версию. Номера присваиваются последовательно: 1, 2, 3, ..., только изменяющим командам.
- Команда может обращаться к **любой уже созданной версии**, включая 0. Исходная версия сохраняется и остаётся доступной.
- Чтение не создаёт версий. Номер версии не совпадает с номером команды.
- Разные номера версий могут обозначать одинаковое содержимое и даже ссылаться на одни и те же узлы.

Полное копирование структуры при каждом изменении не допускается: требуется уложиться в оценки ниже. Для удобства обозначим $L(n) = 1 + \lceil \log_2(n) \rceil$. Это позволяет записывать оценки и при $n = 1$.

1. Стек на списке

Реализуйте обычный стек целых чисел на односвязном списке. Изначально стек пуст. Каждый узел должен хранить значение и указатель на следующий узел. Готовые контейнеры стека или списка использовать нельзя.

Поддержите команды:

- push x - добавить x на вершину;
- pop - удалить вершину и вывести удалённое значение;
- top - вывести значение вершины, не меняя стек.

Команды pop и top даются только для непустого стека. Каждая операция должна работать за $O(1)$. Память для узлов должна составлять $O(s)$, где s - текущий размер стека; удалённые узлы нужно освобождать.

Формат ввода:

```
q
command_1
...
command_q
```

Формат вывода:

Для каждой команды pop или top выведите одно число на отдельной строке. Команда push ничего не выводит. Если запросов на вывод нет, вывод пуст.

Пример

Ввод	Вывод
12	20
push 10	20
push 20	10
top	-5
pop	-5
top	10
push -5	0
top	0
pop	
pop	
push 0	
top	
pop	

На доске: история стека

Пока не пишете код. К пустому стеку версии 0 применили команды:

```
push 0 10
push 1 20
push 1 30
pop 2
push 4 40
pop 3
```

Здесь первый аргумент - исходная версия; результат каждой строки получает следующий номер версии. Нарисуйте отдельно дерево происхождения версий и узлы списков с указателями на вершины всех версий. Одинаковые значения сами по себе не означают, что узел один и тот же.

1. Выпишите содержимое версий 0-6 от вершины к основанию.
2. Отметьте общие узлы. Какие версии имеют один и тот же указатель на вершину?
3. Сколько узлов создано всего? Сколько новых узлов нужно каждой операции?
4. Какие действия из обычного pop теперь недопустимы?

2. Персистентный стек

Переделайте решение задачи 1 так, чтобы оно поддерживало все версии стека. Версия 0 пуста. Поддержите команды:

- push v x - создать версию, добавив x на вершину версии v ;
- pop v - создать версию, удалив вершину версии v ;
- top v - прочитайте значение вершины версии v .

Здесь $0 \leq v \leq k$, где k - число уже созданных версий, не считая начальной. Для pop и top версия v непуста. И push, и pop создают новую версию, даже если её содержимое совпадает с содержимым существующей.

Каждая команда должна работать за $O(1)$. Каждый push создаёт ровно один новый узел списка; pop и top не создают узлов списка. Запись о новой версии учитывается отдельно от узла. Общая память должна быть $O(q)$.

Формат ввода:

```
q
command_1
...
command_q
```

Формат вывода:

- Для push выведите номер созданной версии.
- Для pop выведите номер созданной версии и удалённое значение через пробел.
- Для top выведите прочитанное значение.

Каждой команде соответствует одна строка вывода.

Устно: как добавить запрос size v за $O(1)$? Где можно хранить размер, чтобы не проходить список? Нужно ли создавать узел для пустого стека? Реализовывать этот запрос в данной задаче не требуется.

Пример

Ввод	Вывод
13	1
push 0 10	2
push 1 20	3
push 1 30	20
top 2	4 20
pop 2	5
push 4 40	6 30
pop 3	10
top 1	30
top 3	40
top 5	7 10
pop 6	8
push 7 -1	10
top 6	

3. Массив на дереве отрезков

Дан массив a длины n , $1 \leq n \leq 100000$. Индексы идут от 0 до $n - 1$. Реализуйте его хранение в дереве отрезков **на указателях**. Узел соответствует полуинтервалу $[l, r)$, лист хранит один элемент. Внутренний узел содержит указатели на двух детей. Суммы и другие агрегаты пока не нужны.

Напишите функции `build`, `getElement`, `setElement`. В привычных обозначениях дерева отрезков последние две функции соответствуют точечным `query` и `modify`. Если ваш прежний код хранит дерево в массиве, переделайте его на указатели.

Поддержите команды:

- `get i` - вывести $a[i]$;
- `set i x` - присвоить $a[i] = x$.

В обеих командах $0 \leq i < n$. Изменения выполняются в единственном текущем массиве. Построение должно занимать $O(n)$ времени и памяти, каждая команда - $O(L(n))$ времени. Дополнительные узлы при `set` не нужны.

Формат ввода:

```
n q
a_0 a_1 ... a_(n-1)
command_1
...
command_q
```

Формат вывода:

Для каждой команды `get` выведите одно число на отдельной строке. Команда `set` ничего не выводит. Если команд `get` нет, вывод пуст.

Для отладки можно написать функцию `print`, которая выводит листья слева направо. Она не входит в формат команд; её вывод не должен попадать в ответ. Какова её сложность по сравнению с n вызовами `getElement`?

Пример

Ввод	Вывод
5 10	1
1 2 3 4 5	5
get 0	30
get 4	-1
set 2 30	0
get 2	2
set 0 -1	
get 0	
set 4 0	
get 4	
set 2 30	
get 1	

Место для схем и замечаний

На доске: история массива

Версия 0 содержит [1, 2, 3, 4]. Выполнены команды:

```
set 0 2 30
get 0 2
set 0 0 10
set 1 2 30
```

Первый аргумент - версия, второй - индекс. Новые версии создаёт только set. Используйте дерево, которое делит $[l, r)$ в точке $\text{floor}((l + r) / 2)$.

1. Выпишите номера новых версий и содержимое каждой из них.
2. Нарисуйте корни всех версий и общие поддеревья.
3. Отметьте путь к изменяемому листу. Какие узлы можно оставить общими?
4. Сколько новых узлов потребуется каждому set, если всегда копировать весь путь, даже при присваивании прежнего значения? Сколько создаёт get?
5. Что сломается, если изменить лист в старом узле, а скопировать только корень?

4. Персистентный массив

Переделайте решение задачи 3. Начальный массив - версия 0. Ограничения на n , индексы и значения остаются прежними. Поддержите команды:

- get $v \ i$ - вывести элемент с индексом i в версии v ;
- set $v \ i \ x$ - создать новую версию, отличающуюся от v значением $a[i] = x$.

Здесь $0 \leq v \leq k$, где k - число уже выполненных команд set. Команда get не меняет структуру и не создаёт версию. Команда set создаёт версию даже при присваивании того значения, которое уже записано в элементе.

Используйте дерево на указателях и копирование пути к изменяемому листу. В базовом решении каждый set создаёт по одному узлу на каждом уровне этого пути, включая корень и лист, даже если значение не изменилось. Старые узлы менять нельзя. get не создаёт узлов дерева.

Построение должно занимать $O(n)$ времени. Каждая команда должна работать за $O(L(n))$, каждый set создаёт не более $L(n)$ узлов. Общая память - $O(n + q * L(n))$. Не требуется, чтобы n было степенью двойки.

Формат ввода:

```
n q
a_0 a_1 ... a_(n-1)
command_1
...
command_q
```

Формат вывода:

- Для get выведите прочитанное значение.
- Для set выведите номер созданной версии.

Каждой команде соответствует одна строка вывода.

Объясните, почему память растёт на $O(L(n))$, а не на $O(n)$ при одном изменении.

Пример

Ввод	Вывод
4 12	1
1 2 3 4	3
set 0 2 30	30
get 0 2	2
get 1 2	3
set 0 0 10	3
set 1 2 30	1
get 2 2	4
get 3 0	10
set 2 3 -4	-4
get 4 0	4
get 4 3	4
get 0 3	
get 1 3	

Обсуждение: почему с очередью сложнее?

Очередь добавляет элементы в конец и удаляет из начала. В персистентной очереди и добавление, и удаление создают новую версию.

1. Обычную очередь можно хранить в списке с указателями на начало и конец. Нарисуйте две версии, полученные добавлением разных элементов в одну исходную очередь. Какой общий узел попытались бы изменить обе операции? Достаточно ли скопировать только последний узел?
2. Вспомните очередь на двух стеках: добавляем во входной стек, а при пустом выходном переносим туда весь входной. Почему в обычной очереди суммарное время m операций составляет $O(m)$?
3. Пусть оба стека персистентны, но перенос выполняется целиком и его результат не кешируется. Создайте версию с m элементами во входном стеке и пустым выходным. Затем m раз удалите первый элемент **из этой же версии**, каждый раз создавая новую ветку. Сколько переносов будет выполнено? Получится ли оценка $O(1)$ в среднем по всем выполненным командам?
4. Какие ещё представления обычной очереди вы знаете? Вспомните циклический массив. Что нужно хранить вместе с ним, чтобы отличать пустую очередь от полной?

Продвинутые реализации с постепенным переносом, в том числе конструкции на пяти-шести стеках, оставим для дополнительного чтения после семинара. Для домашнего задания достаточно уже построенного персистентного массива.

Место для схем и замечаний

ДОМАШНЕЕ ЗАДАНИЕ

5. Персистентная очередь на циклическом массиве

Реализуйте персистентную очередь вместимости c , $1 \leq c \leq 100000$. Версия 0 пуста. В каждой отдельной версии одновременно находится не более c элементов, но всего добавлений и версий может быть гораздо больше c .

Используйте циклический массив длины c , представленный персистентным деревом из задачи 4. Вместе с корнем каждой версии храните индекс начала очереди и её размер. Поддержите команды:

- `push v x` - создать версию с элементом x , добавленным в конец версии v ;
- `pop v` - создать версию без первого элемента версии v ;
- `front v` - вывести первый элемент версии v ;
- `size v` - вывести число элементов версии v .

Здесь $0 \leq v \leq k$, где k - число уже выполненных `push` и `pop`. Перед `push` размер исходной версии меньше c . Перед `pop` и `front` она непуста. `size` разрешён и для пустой очереди. При `pop` нужно также вывести удалённый элемент. `push` и `pop` всегда создают новую версию; `front` и `size` не создают.

После построения за $O(c)$ команды `push`, `pop`, `front` должны работать за $O(L(c))$, а `size` - за $O(1)$. В базовом решении только `push` создаёт узлы дерева, не более $L(c)$ штук. Общая память - $O(c + q * L(c))$.

Формат ввода:

```
c q
command_1
...
command_q
```

Формат вывода:

- Для `push` выведите номер созданной версии.
- Для `pop` выведите номер созданной версии и удалённое значение через пробел.
- Для `front` и `size` выведите ответ на запрос.

Каждой команде соответствует одна строка вывода.

Объясните, почему после удаления не нужно очищать ячейку и почему запись в освободившуюся позицию не портит старые версии. Обязательно проверьте переход через конец массива и случай $c = 1$.

Пример

Ввод	Вывод
3 20	1
push 0 10	2
push 1 20	3
push 2 30	4 10
pop 3	5
push 4 40	10
front 3	20
front 5	6 20
pop 5	7 30
pop 6	40
front 7	8 40
pop 7	0
size 8	9
push 8 -5	-5
front 9	10
push 1 99	11 10
pop 10	99
front 11	10
front 2	3
size 5	0
size 0	